

SecureShield

A Role-Based Access Control (RBAC) API

Mini Project II | Information Security | Ostim Technical University | May 2026

1. Why Salting Is Necessary to Prevent Rainbow Table Attacks

When a system stores passwords, it must never store them in plain text. The standard approach is to hash the password using a function like SHA-256 or bcrypt. However, hashing alone introduces a critical vulnerability.

The Problem: If two users have the same password, their hashes will be identical. An attacker can precompute a massive table of common passwords and their hashes — called a Rainbow Table — and instantly look up any matching hash to recover the original password.

Salting solves this by adding a unique random string to each password before hashing it.

```
password = 'mypassword123' salt = bcrypt.gensalt() # unique random
value hash = bcrypt.hashpw(password.encode(), salt)
```

Even if two users have the exact same password, their salts are different, so their hashes will be completely different. This makes precomputed Rainbow Tables useless because the attacker would need to recompute the entire table for every possible salt — which is computationally infeasible.

In SecureShield: every user registration calls `bcrypt.gensalt()` to generate a unique salt, then `bcrypt.hashpw()` to produce the final stored hash. The salt is embedded inside the hash string itself, so no separate storage is needed.

2. Risks of Storing Sensitive Data Inside a JWT Payload

A JSON Web Token (JWT) consists of three parts: a header, a payload, and a signature. The payload is Base64-encoded — not encrypted. This means anyone who possesses the token can decode the payload and read its contents without knowing the secret key.

```
# Decoding a JWT payload requires no secret key: import base64, json payload =
token.split('.')[1] print(json.loads(base64.b64decode(payload + '==')))
```

This creates serious risks if sensitive data is stored in the payload:

- Passwords stored in the payload are exposed to anyone who intercepts or reads the token.
- Personal data such as email addresses, phone numbers, or financial information can be read by any party that handles the token, including browser storage, logs, and proxies.

- Role information can be read by the client, which could reveal the internal permission structure of the system to attackers.

Key Principle: JWT provides integrity — the signature guarantees the payload was not tampered with. But it does not provide confidentiality. Never store sensitive data in a JWT payload.

In SecureShield: the JWT payload only contains the username, role, and expiry time. No passwords, personal data, or secrets are included. Sensitive data remains in the database and is never exposed through the token.

If confidential claims must be transmitted in a token, the correct solution is to use a JWE (JSON Web Encryption) token, which encrypts the payload rather than just signing it.

3. Tool Overview — SecureShield

SecureShield is implemented in a single Python file using FastAPI. The following libraries are used:

- fastapi — web framework for building the API
- bcrypt — for password salting and hashing
- PyJWT — for generating and validating JWT tokens

The API exposes the following endpoints:

```
POST    /register → Register a new user (password hashed with bcrypt) POST    /login
→ Authenticate and receive a JWT token GET      /profile → Accessible by both User
and Admin roles DELETE /user/{id} → Accessible by Admin role only POST    /logout
→ Revoke the current token (blacklisting)
```

Live Demo Summary: A user logs in and receives a token. A standard User attempting to access the DELETE route receives a 403 Forbidden error, which is also logged to security.log. Manually editing the role inside the JWT using jwt.io and re-submitting causes the server to reject the token due to an invalid signature.